

# Real-Time Examples of Inheritance Principle in C#

In this article, I will discuss Multiple **Real-Time Examples of the Inheritance Principle in C#**. Please read our [Inheritance in C#](#) article, where we discussed the basic concepts of Inheritance. At the end of this article, you will understand the following Real-time Examples using the Inheritance Principle in C#.

1. [What is the Inheritance Principle in C#?](#)
2. [Vehicle Management System](#)
3. [Education System](#)
4. [Animals in a Zoo](#)
5. [Electronic Devices](#)
6. [Library System](#)
7. [Computing Devices](#)
8. [Advantages and Disadvantages of Inheritance Principle in C#](#)
9. [When to use Inheritance in C#?](#)

## What is the Inheritance Principle in C#?

Inheritance is one of the core concepts of object-oriented programming (OOP) and plays a significant role in C#. It allows a class (the “derived” or “child” class) to inherit members (like fields, properties, methods, and events) from another class (the “base” or “parent” class). Inheritance promotes code reusability and establishes a hierarchical relationship between classes.

So, inheritance is another core principle of object-oriented programming. It allows a class to inherit properties and methods from another, promoting code reuse and establishing a natural hierarchy between classes. Here’s a breakdown of the key concepts associated with inheritance in C#:

- **Base Class (Parent Class):** The class whose members are inherited by another class. It’s also known as the superclass or parent class.
- **Derived Class (Child Class):** The class that inherits members from the base class. It can also introduce additional members or override inherited ones. It’s also known as the subclass or child class.
- **Inheritance Chain:** Classes can inherit from a class that inherits from another class, leading to an inheritance “chain” or “hierarchy.”
- **Access Modifiers:** Only members with public, protected, or internal access modifiers in the base class are accessible from the derived class. Private members of the base class are not accessible in the derived class.
- **Method Overriding:** If the base class defines a method as virtual, the derived class can override this method using the override keyword. This allows the derived class to provide a specific implementation for that method.
- **The base Keyword:** Inside a derived class, you can use the base keyword to access members of the base class.
- **Sealed Classes:** In C#, if you want to prevent a class from being inherited, you can mark it with the sealed keyword.

## Real-Time Example of Inheritance Principle in C#: Vehicle Management System

Consider different types of vehicles, such as a basic vehicle, a car, and a motorcycle. All vehicles can be started and stopped and have a speed. However, cars and motorcycles have specific properties and

behaviors. Let us see how we can implement this example using the Inheritance Principle in C#:

```
using System;
namespace InheritancePrincipleCSharp
{
    //Base Class (Parent Class) - Vehicle
    public class Vehicle
    {
        public int Speed { get; protected set; }
        public void Start()
        {
            Console.WriteLine("Vehicle started.");
        }
        public void Stop()
        {
            Console.WriteLine("Vehicle stopped.");
        }
        public virtual void Accelerate()
        {
            Speed += 5;
            Console.WriteLine($"Vehicle accelerates. Current speed: {Speed} km/h.");
        }
    }
    //Derived Class (Child Class) - Car
    public class Car : Vehicle
    {
        public int Doors { get; set; }
        public override void Accelerate()
        {
            Speed += 10;
            Console.WriteLine($"Car accelerates. Current speed: {Speed} km/h.");
        }
        public void OpenSunroof()
        {
            Console.WriteLine("Sunroof opened.");
        }
    }
    //Derived Class (Child Class) - Motorcycle
    public class Motorcycle : Vehicle
    {
        public bool HasSideCar { get; set; }
        public override void Accelerate()
        {
            Speed += 7;
            Console.WriteLine($"Motorcycle accelerates. Current speed: {Speed} km/h.");
        }
        public void UseKickstand()
        {
            Console.WriteLine("Kickstand placed.");
        }
    }
}
```

```

}

//Testing Inheritance Principle
public class Program
{
    static void Main(string[] args)
    {
        //Using the Inheritance
        Car myCar = new Car { Doors = 4 };
        myCar.Start();
        myCar.Accelerate();
        myCar.OpenSunroof();
        myCar.Stop();
        Console.WriteLine();
        Motorcycle myBike = new Motorcycle { HasSideCar = false };
        myBike.Start();
        myBike.Accelerate();
        myBike.UseKickstand();
        myBike.Stop();
        Console.Read();
    }
}
}

```

#### In this Example:

- The Vehicle class represents the general properties and behaviors of vehicles.
- The Car and Motorcycle classes inherit from Vehicle, which means they automatically get the Start, Stop, and Accelerate methods. Additionally, they can have their own specific properties (like Doors for Cars or HasSideCar for Motorcycles) and behaviors (like OpenSunroof for Cars or UseKickstand for Motorcycles).
- The Accelerate method is overridden in both Car and Motorcycle to provide specific acceleration behaviors for each.

This example demonstrates the power of inheritance to promote code reuse, establish hierarchies, and allow specialized behavior in derived classes. When you run the above code, you will get the following output:

```

Vehicle started.
Car accelerates. Current speed: 10 km/h.
Sunroof opened.
Vehicle stopped.

Vehicle started.
Motorcycle accelerates. Current speed: 7 km/h.
Kickstand placed.
Vehicle stopped.

```

## Real-Time Example of Inheritance Principle in C#: Education System

Let's consider another real-time example to understand the inheritance principle in C#, i.e., the education system scenario with students and teachers. In an educational institution, students and teachers are people with common attributes like name, age, and address. However, they also have distinct properties and behaviors. For instance, a student may enroll in a course while a teacher might teach one. Let us see how we can implement this example using the Inheritance Principle in C#:

```
using System;
namespace InheritancePrincipleCSharp
{
    //Base Class (Parent Class) - Person
    public class Person
    {
        public string Name { get; set; }
        public int Age { get; set; }
        public string Address { get; set; }
        public Person(string name, int age, string address)
        {
            Name = name;
            Age = age;
            Address = address;
        }
        public void DisplayDetails()
        {
            Console.WriteLine($"Name: {Name}, Age: {Age}, Address: {Address}");
        }
    }
    //Derived Class (Child Class) - Student
    public class Student : Person
    {
        public string StudentId { get; set; }
        public Student(string name, int age, string address, string studentId)
            : base(name, age, address) // Calling base class constructor
        {
            StudentId = studentId;
        }
        public void Enroll(string courseName)
        {
            Console.WriteLine($"{Name} has enrolled in {courseName} course.");
        }
    }
    //Derived Class (Child Class) - Teacher
    public class Teacher : Person
    {
        public string EmployeeId { get; set; }
        public Teacher(string name, int age, string address, string employeeId)
            : base(name, age, address) // Calling base class constructor
        {
            EmployeeId = employeeId;
        }
    }
}
```

```

    }
    public void Teach(string courseName)
    {
        Console.WriteLine($"{Name} is teaching {courseName} course.");
    }
}

//Testing Inheritance Principle
public class Program
{
    static void Main(string[] args)
    {
        //Using the Inheritance
        Student john = new Student("John Doe", 20, "123 Main St", "S12345");
        john.DisplayDetails();
        john.Enroll("Mathematics");
        Console.WriteLine();
        Teacher mrsSmith = new Teacher("Mrs. Smith", 40, "456 Elm St", "T67890");
        mrsSmith.DisplayDetails();
        mrsSmith.Teach("Physics");
        Console.Read();
    }
}
}

```

#### In this example:

- The Person class captures general attributes and behaviors common to students and teachers.
- The Student and Teacher classes inherit from Person. This inheritance means they automatically have properties like Name, Age, and Address and the DisplayDetails method. Additionally, they introduce specific properties and methods like Enroll for Student and Teach for Teacher.
- The derived classes' constructors utilize the base keyword to invoke the base class's constructor, ensuring that the common properties are set.

This example demonstrates how inheritance can model real-world relationships, facilitate code reuse, and provide a structured way to represent hierarchies in your system.

## Real-Time Example of Inheritance Principle in C#: Animals in a Zoo

A zoo has different types of animals, like mammals, birds, and reptiles. All animals have common properties such as name, age, and diet, but each type might have unique behaviors. For instance, birds can fly, while mammals might have a specific communication method. Let us see how we can implement this example using the Inheritance Principle in C#:

```

using System;
namespace InheritancePrincipleCSharp
{
    //Base Class (Parent Class) - Animal
    public class Animal
    {

```

```

    public string Name { get; set; }
    public int Age { get; set; }
    public string Diet { get; set; }
    public Animal(string name, int age, string diet)
    {
        Name = name;
        Age = age;
        Diet = diet;
    }
    public void Eat()
    {
        Console.WriteLine($"{Name} is eating {Diet}.");
    }
    public virtual void Display()
    {
        Console.WriteLine($"I am {Name}, a {Age}-year-old animal that eats {Diet}.");
    }
}
//Derived Class (Child Class) - Bird
public class Bird : Animal
{
    public bool CanFly { get; set; }
    public Bird(string name, int age, string diet, bool canFly)
        : base(name, age, diet)
    {
        CanFly = canFly;
    }
    public void Fly()
    {
        if (CanFly)
            Console.WriteLine($"{Name} is flying.");
        else
            Console.WriteLine($"{Name} cannot fly.");
    }
    public override void Display()
    {
        base.Display();
        Fly();
    }
}
//Derived Class (Child Class) - Mammal
public class Mammal : Animal
{
    public string CommunicationSound { get; set; }
    public Mammal(string name, int age, string diet, string sound)
        : base(name, age, diet)
    {
        CommunicationSound = sound;
    }
    public void Communicate()
    {

```

```

        Console.WriteLine($"{Name} makes a {CommunicationSound} sound.");
    }
    public override void Display()
    {
        base.Display();
        Communicate();
    }
}

//Testing Inheritance Principle
public class Program
{
    static void Main(string[] args)
    {
        //Using the Inheritance
        Bird parrot = new Bird("Parrot", 5, "seeds", true);
        parrot.Display();
        Console.WriteLine();
        Mammal lion = new Mammal("Lion", 8, "meat", "roar");
        lion.Display();
        Console.Read();
    }
}
}

```

#### In this example:

- The Animal class represents general properties and behaviors common to all animals.
- The Bird and Mammal classes inherit from Animals, implying they automatically obtain properties like Name, Age, Diet, and the Eat method. However, they also introduce specific behaviors: Bird has a Fly method, while Mammal has a Communicate method.
- The Display method is overridden in both Bird and Mammal classes to provide specialized behavior in addition to the base behavior.

Through this model, inheritance helps effectively represent a natural hierarchy of animals and promotes code reuse.

## Real-Time Example of Inheritance Principle in C#: Electronic Devices

Consider various electronic devices, like a basic mobile phone and a smartphone. While all electronic devices can be powered on or off, smartphones have additional capabilities, such as internet access. Let us see how we can implement this example using the Inheritance Principle in C#:

```

using System;
namespace InheritancePrincipleCSharp
{
    //Base Class (Parent Class) - ElectronicDevice
    public class ElectronicDevice
    {
        public string Brand { get; set; }
    }
}

```

```

    public void PowerOn()
    {
        Console.WriteLine($"{Brand} device is powered on.");
    }
    public void PowerOff()
    {
        Console.WriteLine($"{Brand} device is powered off.");
    }
}
//Derived Class (Child Class) - MobilePhone
public class MobilePhone : ElectronicDevice
{
    public void MakeCall(string number)
    {
        Console.WriteLine($"Calling {number} from {Brand} mobile phone.");
    }
    public void ReceiveCall(string number)
    {
        Console.WriteLine($"Receiving call from {number} on {Brand} mobile");
    }
}
//Derived Class (Child Class) - SmartPhone
public class SmartPhone : MobilePhone
{
    public void BrowseWeb(string website)
    {
        Console.WriteLine($"Browsing {website} on {Brand} smartphone.");
    }
    public void InstallApp(string appName)
    {
        Console.WriteLine($"Installing {appName} on {Brand} smartphone.");
    }
}

//Testing Inheritance Principle
public class Program
{
    static void Main(string[] args)
    {
        //Using the Inheritance
        MobilePhone nokia3310 = new MobilePhone { Brand = "Nokia" };
        nokia3310.PowerOn();
        nokia3310.MakeCall("123-456-7890");
        nokia3310.ReceiveCall("098-765-4321");
        nokia3310.PowerOff();
        Console.WriteLine();
        SmartPhone iPhone = new SmartPhone { Brand = "Apple" };
        iPhone.PowerOn();
        iPhone.BrowseWeb("www.example.com");
        iPhone.InstallApp("ChatApp");
        iPhone.MakeCall("123-456-7890");
    }
}

```

```

        iPhone.PowerOff();
        Console.Read();
    }
}
}

```

**In this example:**

- The ElectronicDevice class encapsulates behaviors common to all electronic devices.
- The MobilePhone class, while inheriting general properties from ElectronicDevice, introduces basic phone capabilities such as making and receiving calls.
- The SmartPhone class, derived from MobilePhone, not only inherits the phone capabilities but also introduces advanced features like browsing the web and installing apps.

The inheritance chain (ElectronicDevice -> MobilePhone -> SmartPhone) illustrates the progressive specialization of classes. This hierarchy promotes code reuse and establishes a clear relationship between the classes.

## Real-Time Example of Inheritance Principle in C#: Library System

A library contains various items, including books, magazines, and DVDs. All library items have a unique identifier and a title and can be borrowed or returned. However, each type might have unique properties; for instance, a book has an author and many pages, while a DVD might have a runtime. Let us see how we can implement this example using the Inheritance Principle in C#:

```

using System;
namespace InheritancePrincipleCSharp
{
    //Base Class (Parent Class) - LibraryItem
    public class LibraryItem
    {
        public string Id { get; set; }
        public string Title { get; set; }
        public LibraryItem(string id, string title)
        {
            Id = id;
            Title = title;
        }
        public void Borrow()
        {
            Console.WriteLine($"'{Title}' has been borrowed.");
        }
        public void Return()
        {
            Console.WriteLine($"'{Title}' has been returned.");
        }
    }
    //Derived Class (Child Class) - Book
    public class Book : LibraryItem
    {

```

```

    public string Author { get; set; }
    public int Pages { get; set; }
    public Book(string id, string title, string author, int pages)
        : base(id, title)
    {
        Author = author;
        Pages = pages;
    }
    public void DisplayBookInfo()
    {
        Console.WriteLine($"Book: '{Title}' by {Author}, {Pages} pages.");
    }
}
//Derived Class (Child Class) - DVD
public class DVD : LibraryItem
{
    public int Runtime { get; set; } // Runtime in minutes
    public DVD(string id, string title, int runtime)
        : base(id, title)
    {
        Runtime = runtime;
    }
    public void DisplayDVDInfo()
    {
        Console.WriteLine($"DVD: '{Title}', Runtime: {Runtime} minutes.");
    }
}

//Testing Inheritance Principle
public class Program
{
    static void Main(string[] args)
    {
        //Using the Inheritance
        Book novel = new Book("BK001", "The Great Novel", "John Doe", 320);
        novel.DisplayBookInfo();
        novel.Borrow();
        novel.Return();
        Console.WriteLine();
        DVD movie = new DVD("DV001", "Epic Movie", 120);
        movie.DisplayDVDInfo();
        movie.Borrow();
        movie.Return();
        Console.Read();
    }
}
}

```

**In this Example:**

- The `LibraryItem` class defines basic properties and behaviors common to all library items, such as borrowing and returning.
- The `Book` and `DVD` classes inherit from `LibraryItem`, automatically obtaining properties like `Id` and `Title` and methods like `Borrow` and `Return`. Yet, they also add their own unique attributes: `Book` introduces `Author` and `Pages`, while `DVD` introduces `Runtime`.
- Each class has a method to display specific details: `DisplayBookInfo` for `Book` and `DisplayDVDInfo` for `DVD`.

Through this example, the inheritance principle in C# allows us to model a real-world library system, promote code reuse, and maintain an organized structure of the different items within the library.

## Real-Time Example of Inheritance Principle in C#: Computing Devices

A tech company manufactures various computing devices, including desktops, laptops, and tablets. All these devices share some basic attributes, like processor, RAM, and storage capacity. However, each device type can have its unique attributes and behaviors. For example, a laptop has battery life, while a desktop might have the type of cooling system it uses. Let us see how we can implement this example using the Inheritance Principle in C#:

```
using System;
namespace InheritancePrincipleCSharp
{
    //Base Class (Parent Class) - Device
    public class Device
    {
        public string Processor { get; set; }
        public int RAM { get; set; } // in GB
        public int Storage { get; set; } // in GB
        public Device(string processor, int ram, int storage)
        {
            Processor = processor;
            RAM = ram;
            Storage = storage;
        }
        public void BootUp()
        {
            Console.WriteLine("Device is booting up...");
        }
    }
    //Derived Class (Child Class) - Desktop
    public class Desktop : Device
    {
        public string CoolingSystem { get; set; }
        public Desktop(string processor, int ram, int storage, string coolingSystem)
            : base(processor, ram, storage)
        {
            CoolingSystem = coolingSystem;
        }
        public void DisplayDesktopInfo()
        {

```

```

        Console.WriteLine($"Desktop with {Processor}, {RAM}GB RAM, {Storage}GB Storage, and {CoolingSystem} cooling.");
    }
}
//Derived Class (Child Class) - Laptop
public class Laptop : Device
{
    public int BatteryLife { get; set; } // in hours
    public Laptop(string processor, int ram, int storage, int batteryLife)
        : base(processor, ram, storage)
    {
        BatteryLife = batteryLife;
    }
    public void DisplayLaptopInfo()
    {
        Console.WriteLine($"Laptop with {Processor}, {RAM}GB RAM, {Storage}GB Storage, and {BatteryLife} hours battery life.");
    }
}

//Testing Inheritance Principle
public class Program
{
    static void Main(string[] args)
    {
        //Using the Inheritance
        Desktop gamingPC = new Desktop("Intel i9", 32, 1024, "Liquid Cooling");
        gamingPC.DisplayDesktopInfo();
        gamingPC.BootUp();
        Console.WriteLine();
        Laptop ultrabook = new Laptop("Intel i7", 16, 512, 10);
        ultrabook.DisplayLaptopInfo();
        ultrabook.BootUp();
        Console.Read();
    }
}
}

```

### In this Example:

- The base class Device provides a foundation with general properties and behaviors relevant to all computing devices.
- The derived classes Desktop and Laptop inherit the general properties and behaviors from Device and also introduce their specific properties, such as CoolingSystem for desktops and BatteryLife for laptops.

Through this hierarchy, the inheritance principle in C# enables the representation of a real-world relationship between different computing devices, promoting code reusability and maintaining an organized structure.

## Advantages and Disadvantages of Inheritance Principle in C#

Inheritance is a fundamental concept in object-oriented programming (OOP) and offers a range of benefits, but it also has some disadvantages. Here are the advantages and disadvantages of the inheritance principle in C#:

### Advantages of Inheritance Principle in C#:

- **Code Reusability:** One of the most significant benefits of inheritance is the ability to reuse code from the base class in the derived class. This reduces redundancy and leads to smaller, more maintainable codebases.
- **Extensibility:** Derived classes can introduce new attributes and behaviors or modify the inherited ones, allowing for easy extensions to the functionality without modifying the base class.
- **Hierarchical Organization:** Inheritance helps organize classes hierarchically, which mirrors real-world relationships and hierarchies, making the design more intuitive.
- **Polymorphism:** Inheritance is a prerequisite for implementing type polymorphism in OOP. Polymorphism allows objects of different classes to be treated as objects of a common superclass, facilitating flexibility in using objects.
- **Standardization:** By defining standard methods and properties in the base class, you ensure that all derived classes have a consistent interface, making it easier for other developers to understand and work with the derived classes.

### Disadvantages of Inheritance Principle in C#:

- **Overhead:** Unnecessary inheritance (when not needed) can introduce additional overhead regarding memory and processing because the derived class will carry the baggage of the base class's attributes and methods.
- **Complexity:** Deep and extensive inheritance hierarchies can become hard to manage and understand. This can make the code more error-prone and harder to maintain.
- **Tight Coupling:** Inheritance results in tight coupling between the base and derived classes. Changes in the base class can have ripple effects on all derived classes, which might lead to unintended consequences and more fragile software.
- **Inheritance Limitation:** C# does not support multiple inheritances for classes (i.e., a class cannot inherit from more than one class). This limitation might force developers to use interfaces or other design patterns when they need to inherit behaviors from multiple sources.
- **Overuse:** Some developers overuse inheritance by creating long chains even when composition or other design principles might be more appropriate. This can lead to inflexible designs where not all base class properties and methods make sense for all derived classes.

While inheritance is a powerful tool in OOP and C#. Developers should understand when to use inheritance and when to opt for other design principles like composition or aggregation. It's always important to consider the long-term implications of using inheritance on the code's maintainability, extensibility, and robustness.

### When to use Inheritance in C#?

Here are some situations where it makes sense to use inheritance in C#:

- **Is-A Relationship:** Inheritance is most suitable when a clear "Is-A" relationship exists between the base class and the derived class. For example, a Dog is an Animal, so it makes sense to inherit from an Animal.
- **Code Reusability:** When multiple classes share common attributes and behaviors, these commonalities can be abstracted into a base class to avoid code duplication. For instance, if various

bank accounts share some properties and methods, they can inherit from a general BankAccount class.

- **Polymorphism:** Inheritance is required if you want to leverage polymorphism, where you can refer to objects of derived classes using a reference of the base class and execute overridden methods. This is useful in scenarios where you must process a list of objects that share a common base class but might belong to different derived classes.
- **Extensibility:** If you're designing a library or framework and want to allow users to extend or customize certain functionalities, defining a base class that users can inherit from and override can be beneficial.
- **Standardization:** When you want to ensure that a group of classes has a standardized set of attributes or methods, defining them in a base class ensures that all derived classes will have them.

However, there are also cases where inheritance might not be the best choice:

- **No Clear "Is-A" Relationship:** If the relationship between classes is more of a "Has-A" rather than an "Is-A," composition is generally a better choice. For instance, a Car has an Engine, but a Car isn't an Engine.
- **Avoid Deep Inheritance Hierarchies:** Deep and extensive inheritance chains can become hard to manage and make the code error-prone. Prefer shallow inheritance trees and consider composition or interfaces for more complex scenarios.
- **Avoid Overhead:** It might be overkill if you're inheriting just for a few properties or methods. Unnecessary inheritance can introduce extra overhead and lead to less flexible designs.
- **Multiple Inheritance:** Since C# doesn't support multiple inheritance for classes, you might want to use interfaces instead if you need to inherit behaviors from multiple sources.
- **Tight Coupling Concerns:** If you're concerned about changes in the base class affecting derived classes or if the base class is frequently changing, it might be better to use composition or other patterns to decouple the classes.

In the next article, I will discuss the [Multiple Real-time Examples of the Polymorphism Principle in C#](#). In this article, I explain **Real-Time Examples of the Inheritance Principle in C#**. I hope you enjoy this Inheritance Principle in Real-Time Examples using the C# article.